Prétraitement et comparaison des méthodes d'imputation des valeurs manquantes

BODJRENOU Marcellin & AKODOHOU Ginette

Table of contents

1 Introduction **2**

2 Apercu genérale des données reçu **2**
2.1 Importation des bibliothèques . . . 2
2.2 Chargement des données reçu . . . 3
2.3 Aperçu des données reçu . . . 3
2.4 Informations générales . . . 3
2.5 Les variables . . . 5
2.6 Stations présentes dans les données . . . 5
2.7 Les types de donné reçu . . . 5

3 Création des jeux de données pour le projet **6**
3.1 Création des variables année, mois et jour de Kandi et de Parakou . . . 6
3.2 Aperçu du dataset de Kandi . . . 6
3.3 Aperçu du dataset de Parakou . . . 7
3.4 Création des variables Pression, température, vitesse de vent, humidité et pluviométrie . . . 7
3.5 Aperçu final du dataset de Kandi . . . 9
3.6 Aperçu final du dataset de Parakou . . . 10
3.7 Dimensions des datasets . . . 11
3.8 Sauvegarde des datasets . . . 11

4 Prétraitement des données **11**
4.1 Valeurs manquantes de Kandi . . . 11
4.2 Valeurs manquantes de Parakou . . . 11
4.3 Visualisation des valeurs manquantes de Kandi . . . 12
4.4 Visualisation des valeurs manquantes de Parakou . . . 12
4.5 Suppression des lignes très incomplètes . . . 13

4.6 Méthodes d'imputation . . . 14
4.6.1 Moyenne historique et médiane historique . . . 14
4.6.2 Interpolation linéaire . . . 14
4.6.3 Moyenne globale et médiane globale . . . 14
4.7 Protocole expérimental . . . 15
4.8 Résultats de Kandi . . . 15
4.9 Résultats de Parakou . . . 15
4.10 Représentation graphique des erreurs . . . 16
4.10.1 Station de Kandi . . . 16
4.10.2 Station de Parakou . . . 17

5 Conclusion **18**

6 Prétaitement final des données avec la méthode d'interpolation linéaire **19**

1 Introduction

Dans le cadre de cette étude portant sur l'analyse et la prévision des données météorologiques des stations de Kandi et de Parakou sur la période 2015-2025, une étape importante de prétraitement des données a été réalisée.

L'objectif de ce travail est :

- de créer des jeux de données spécifiques à chaque station ;
- d'analyser les valeurs manquantes ;
- de comparer plusieurs méthodes d'imputation ;
- de sélectionner la méthode la plus performante.

2 Apercu genérale des données reçu

2.1 Importation des bibliothèques

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

import missingno as msn
```

```
from sklearn.metrics import mean_absolute_error,mean_squared_error
```

2.2 Chargement des données reçu

```
data=pd.read_excel("Donnees_Stagiaire_Marcellin.xls")
```

2.3 Aperçu des données reçu

```
print(data.head())
```

```
             Name  Geogr1   Geogr2  Year Eg el abbreviation  \
0  KANDI_AEROPORT  2.9333  11.1333  2015              PSTAD
1  KANDI_AEROPORT  2.9333  11.1333  2015              PSTAD
2  KANDI_AEROPORT  2.9333  11.1333  2015              PSTAD
3  KANDI_AEROPORT  2.9333  11.1333  2015              PSTAD
4  KANDI_AEROPORT  2.9333  11.1333  2015              PSTAD

                Name.1 Description  Month   Time     01  ...     22     23  \
0  Pression Station Moy         hPa      1  06:00  979.9  ...  979.6  978.7
1  Pression Station Moy         hPa      2  06:00  977.7  ...  977.4  979.5
2  Pression Station Moy         hPa      3  06:00  978.6  ...  974.8  974.3
3  Pression Station Moy         hPa      4  06:00  978.5  ...  975.9  975.9
4  Pression Station Moy         hPa      5  06:00  973.9  ...  977.8  976.9

      24     25     26     27     28     29     30     31
0  977.0  976.4  977.9  978.3  978.1  977.9  978.3  978.3
1  979.2  976.8  976.5  978.2  978.4    NaN    NaN    NaN
2  974.1  975.4  977.9  978.5  978.1  976.5  976.2  977.1
3  975.9  977.5  977.2  977.1  976.2  976.0  974.7    NaN
4  977.9  977.0  975.7  976.2  976.6  975.5  976.4  976.2

[5 rows x 40 columns]
```

2.4 Informations générales

```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1577 entries, 0 to 1576
Data columns (total 40 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Name                 1577 non-null   str
 1   Geogr1               1577 non-null   float64
 2   Geogr2               1577 non-null   float64
 3   Year                 1577 non-null   int64
 4   Eg el abbreviation   1577 non-null   str
 5   Name.1               1577 non-null   str
 6   Description          1577 non-null   str
 7   Month                1577 non-null   int64
 8   Time                 1577 non-null   str
 9   01                   1576 non-null   float64
 10  02                   1575 non-null   float64
 11  03                   1574 non-null   float64
 12  04                   1576 non-null   float64
 13  05                   1575 non-null   float64
 14  06                   1576 non-null   float64
 15  07                   1576 non-null   float64
 16  08                   1576 non-null   float64
 17  09                   1576 non-null   float64
 18  10                   1576 non-null   float64
 19  11                   1576 non-null   float64
 20  12                   1577 non-null   float64
 21  13                   1575 non-null   float64
 22  14                   1576 non-null   float64
 23  15                   1577 non-null   float64
 24  16                   1577 non-null   float64
 25  17                   1576 non-null   float64
 26  18                   1577 non-null   float64
 27  19                   1577 non-null   float64
 28  20                   1577 non-null   float64
 29  21                   1577 non-null   float64
 30  22                   1574 non-null   float64
 31  23                   1574 non-null   float64
 32  24                   1576 non-null   float64
 33  25                   1576 non-null   float64
```

```
 34  26                  1576 non-null   float64
 35  27                  1571 non-null   float64
 36  28                  1576 non-null   float64
 37  29                  1480 non-null   float64
 38  30                  1435 non-null   float64
 39  31                  906 non-null    float64
dtypes: float64(33), int64(2), str(5)
memory usage: 492.9 KB
```

2.5 Les variables

```
data.columns
```

```
Index(['Name', 'Geogr1', 'Geogr2', 'Year', 'Eg el abbreviation', 'Name.1',
       'Description', 'Month', 'Time', '01', '02', '03', '04', '05', '06',
       '07', '08', '09', '10', '11', '12', '13', '14', '15', '16', '17', '18',
       '19', '20', '21', '22', '23', '24', '25', '26', '27', '28', '29', '30',
       '31'],
      dtype='str')
```

2.6 Stations présentes dans les données

```
data.Name.unique()
```

```
<StringArray>
['KANDI_AEROPORT', 'PARAKOU_AEROPORT']
Length: 2, dtype: str
```

2.7 Les types de donné reçu

```
print(data["Name.1"].unique())

print(data.Description.unique())
```

```
<StringArray>
['Pression Station Moy',           'Pluie jour',      'Température max',
      'Température min',         'Humidité moy',       'Vit moy vent10']
Length: 6, dtype: str
<StringArray>
['hPa', 'mm', '°C', '%', 'm/s']
Length: 5, dtype: str
```

3 Création des jeux de données pour le projet

Les données fournies constituent un dataframe dans lequel chaque ligne représente une information précise (pression, température, humidité, etc.) des 31 jours d'un mois pour une année donnée.

3.1 Création des variables année, mois et jour de Kandi et de Parakou

```
temps=pd.date_range(start='2015-01-01',end='2025-12-31',freq='D')

data_Kandi=pd.DataFrame({"Lieu":"KANDI_AEROPORT","annee":temps.year,
                         "mois":temps.month,"jour":temps.day})

data_parakou=pd.DataFrame({"Lieu":"PARAKOU_AEROPORT","annee":temps.year,
                         "mois":temps.month,"jour":temps.day})
```

3.2 Aperçu du dataset de Kandi

```
data_Kandi
```

	Lieu	annee	mois	jour
0	KANDI_AEROPORT	2015	1	1
1	KANDI_AEROPORT	2015	1	2
2	KANDI_AEROPORT	2015	1	3
3	KANDI_AEROPORT	2015	1	4
4	KANDI_AEROPORT	2015	1	5
...	...	...	...	...
4013	KANDI_AEROPORT	2025	12	27

	Lieu	annee	mois	jour
4014	KANDI_AEROPORT	2025	12	28
4015	KANDI_AEROPORT	2025	12	29
4016	KANDI_AEROPORT	2025	12	30
4017	KANDI_AEROPORT	2025	12	31

3.3 Aperçu du dataset de Parakou

```
data_parakou
```

	Lieu	annee	mois	jour
0	PARAKOU_AEROPORT	2015	1	1
1	PARAKOU_AEROPORT	2015	1	2
2	PARAKOU_AEROPORT	2015	1	3
3	PARAKOU_AEROPORT	2015	1	4
4	PARAKOU_AEROPORT	2015	1	5
...	...	...	...	...
4013	PARAKOU_AEROPORT	2025	12	27
4014	PARAKOU_AEROPORT	2025	12	28
4015	PARAKOU_AEROPORT	2025	12	29
4016	PARAKOU_AEROPORT	2025	12	30
4017	PARAKOU_AEROPORT	2025	12	31

3.4 Création des variables Pression, température, vitesse de vent, humidité et pluviométrie

```
data=data.rename(columns={"01":"1","02":"2","03":"3","04":"4","05":"5",
                "06":"6","07":"7","08":"8","09":"9"})
def verif(data_verif):
    if 0<len(data_verif):
        return data_verif.iloc[0]
    else:
        return np.nan
def verif_temp(data_verif_temp):
    if 2<=len(data_verif_temp):
        return [data_verif_temp.iloc[0],data_verif_temp.iloc[1]]
```

```
    elif len(data_verif_temp)==1:
        return [data_verif_temp.iloc[0],data_verif_temp.iloc[0]]
    else:
        return[np.nan,np.nan]

def creation(data_ge,lieu):
    list_pression=[]
    list_pluvimetrie=[]
    list_temperature_max=[]
    list_temperature_min=[]
    list_humidite=[]
    list_vitesse_vent=[]

    data_gene=data_ge.copy()
    for i in range(len(data_gene)):
        Annee=data_gene.iloc[i]["annee"]
        Mois=data_gene.iloc[i]["mois"]
        Jour=data_gene.iloc[i]["jour"]

        data_virt=data[(data.Name==lieu) & (data.Year==Annee) & (data.Month==Mois)]

        list_pression.append(verif(data_virt[data_virt.Description=="hPa"][f"{Jour}"]))

        list_temperature_max.append(
            verif_temp(data_virt[data_virt.Description=='°C'][f"{Jour}"])[0])

        list_temperature_min.append(
            verif_temp(data_virt[data_virt.Description=='°C'][f"{Jour}"])[1])

        list_humidite.append(
            verif(data_virt[data_virt.Description=='%'][f"{Jour}"]))

        list_vitesse_vent.append(
            verif(data_virt[data_virt.Description=='m/s'][f"{Jour}"]))

        list_pluvimetrie.append(
            verif(data_virt[data_virt.Description=="mm"][f"{Jour}"]))

    data_gene["Pression"]=list_pression

    data_gene["Temperature_Max"]=list_temperature_max
```

```
    data_gene["Temperature_Min"]=list_temperature_min

    data_gene["Humidite"]=list_humidite

    data_gene["Vitesse_Vent"]=list_vitesse_vent

    data_gene["Pluviometrie"]=list_pluvimetrie

    return data_gene

Kandi=creation(data_Kandi,"KANDI_AEROPORT")

Parakou=creation(data_parakou,"PARAKOU_AEROPORT")
```

3.5 Aperçu final du dataset de Kandi

```
print(Kandi)
```

```
                Lieu  annee  mois  jour  Pression  Temperature_Max  \
0     KANDI_AEROPORT   2015     1     1     979.9             32.0
1     KANDI_AEROPORT   2015     1     2     981.4             30.3
2     KANDI_AEROPORT   2015     1     3     982.8             29.2
3     KANDI_AEROPORT   2015     1     4     983.8             29.0
4     KANDI_AEROPORT   2015     1     5     982.7             29.5
...              ...    ...   ...   ...       ...              ...
4013  KANDI_AEROPORT   2025    12    27     976.9             37.4
4014  KANDI_AEROPORT   2025    12    28     977.5             36.9
4015  KANDI_AEROPORT   2025    12    29     978.2             37.5
4016  KANDI_AEROPORT   2025    12    30     978.9             37.7
4017  KANDI_AEROPORT   2025    12    31     978.4             37.3

      Temperature_Min  Humidite  Vitesse_Vent  Pluviometrie
0                13.4      32.0           2.4           0.0
1                17.4      26.0           2.3           0.0
2                17.7      20.0           3.5           0.0
3                15.8      20.0           2.8           0.0
4                13.5      29.0           3.0           0.0
...               ...       ...           ...           ...
4013             20.4      28.0           1.4           0.0
4014             19.7      27.0           2.0           0.0
```

```
4015             19.8      31.0           1.5           0.0
4016             20.0      32.0           1.9           0.0
4017             19.7      30.0           2.1           0.0

[4018 rows x 10 columns]
```

3.6 Aperçu final du dataset de Parakou

```
print(Parakou)
```

```
                 Lieu  annee  mois  jour  Pression  Temperature_Max  \
0     PARAKOU_AEROPORT   2015     1     1     967.5             34.6
1     PARAKOU_AEROPORT   2015     1     2     968.6             32.6
2     PARAKOU_AEROPORT   2015     1     3     969.9             32.5
3     PARAKOU_AEROPORT   2015     1     4     970.8             32.5
4     PARAKOU_AEROPORT   2015     1     5     969.9             31.9
...                ...    ...   ...   ...       ...              ...
4013  PARAKOU_AEROPORT   2025    12    27     966.0             36.0
4014  PARAKOU_AEROPORT   2025    12    28     966.3             37.2
4015  PARAKOU_AEROPORT   2025    12    29     967.2             36.0
4016  PARAKOU_AEROPORT   2025    12    30     968.0             36.3
4017  PARAKOU_AEROPORT   2025    12    31     967.2             37.2

      Temperature_Min  Humidite  Vitesse_Vent  Pluviometrie
0                19.2      23.0           3.6           0.0
1                18.5      24.0           4.3           0.0
2                18.9      20.0           4.9           0.0
3                16.5      23.0           5.0           0.0
4                15.9      23.0           5.1           0.0
...               ...       ...           ...           ...
4013             22.3      61.0           1.6           0.0
4014             22.2      56.0           2.3           0.0
4015             21.2      64.0           2.1           0.0
4016             22.7      65.0           2.3           0.0
4017             23.1      55.0           2.4           0.0

[4018 rows x 10 columns]
```

3.7 Dimensions des datasets

```
print("Dimension Kandi :", Kandi.shape)

print("Dimension Parakou :", Parakou.shape)
```

```
Dimension Kandi : (4018, 10)
Dimension Parakou : (4018, 10)
```

3.8 Sauvegarde des datasets

```
Kandi.to_csv("data/Kandi.csv",index=False)

Parakou.to_csv("data/Parakou.csv",index=False)
```

4 Prétraitement des données

4.1 Valeurs manquantes de Kandi

```
Kandi.isnull().sum()[0<Kandi.isnull().sum()]
```

```
Pression        66
Humidite         5
Vitesse_Vent    31
Pluviometrie     1
dtype: int64
```

4.2 Valeurs manquantes de Parakou

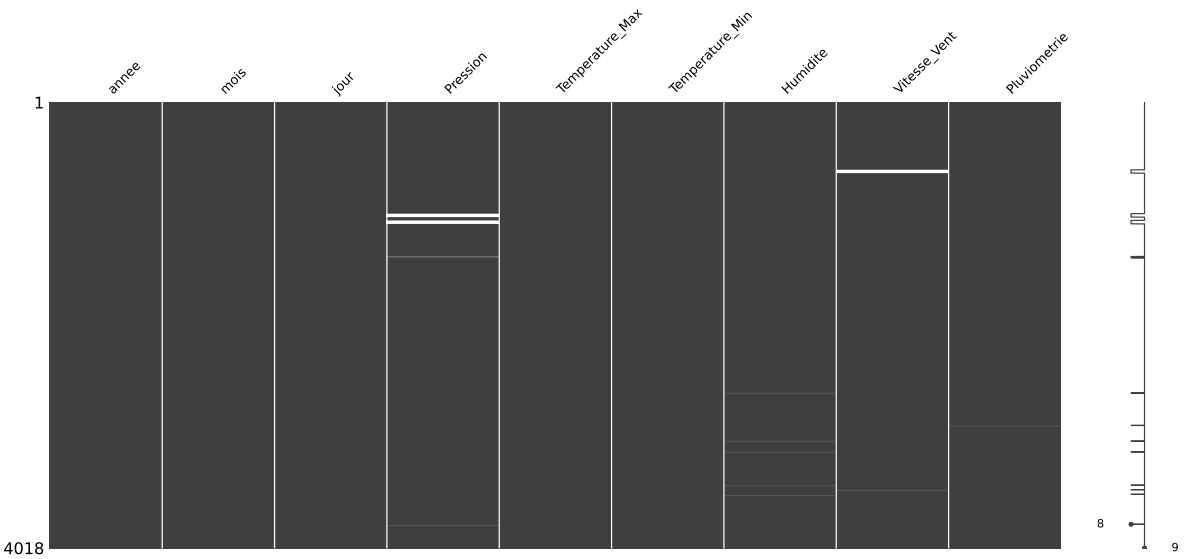
```
Parakou.isnull().sum()[0<Parakou.isnull().sum()]
```

```
Pression           72
Temperature_Max    25
Temperature_Min     4
Humidite           36
Vitesse_Vent        4
dtype: int64
```

4.3 Visualisation des valeurs manquantes de Kandi

```
plt.figure(figsize=(10,8))
msn.matrix(Kandi.select_dtypes(include="number"))
```

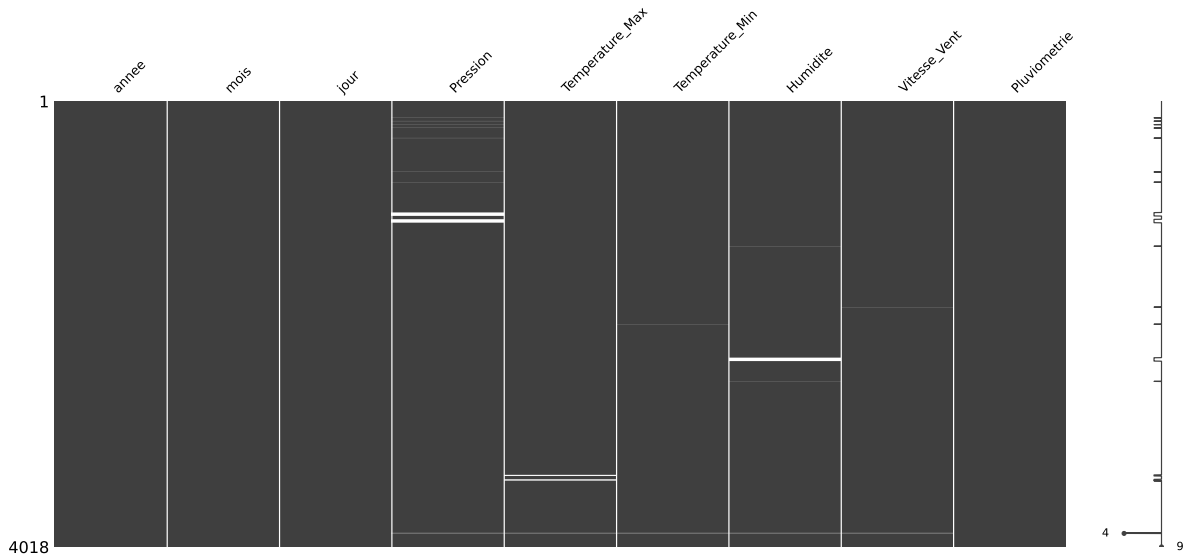
```
<Figure size 3000x2400 with 0 Axes>
```



4.4 Visualisation des valeurs manquantes de Parakou

```
plt.figure(figsize=(10,8),dpi=100)
msn.matrix(Parakou.select_dtypes(include="number"))
```

```
<Figure size 1000x800 with 0 Axes>
```



4.5 Suppression des lignes très incomplètes

```
print(Parakou[(Parakou.Pression.isnull()) & (Parakou.Humidite.isnull()) &
        (Parakou.Temperature_Max.isnull()) & (Parakou.Temperature_Min.isnull())
        &(Parakou.Vitesse_Vent.isnull())])

print(Kandi[Kandi.Pluviometrie.isnull()])
```

```
                Lieu  annee  mois  jour  Pression  Temperature_Max  \
3891  PARAKOU_AEROPORT   2025     8    27       NaN              NaN
3894  PARAKOU_AEROPORT   2025     8    30       NaN              NaN
3895  PARAKOU_AEROPORT   2025     8    31       NaN              NaN

      Temperature_Min  Humidite  Vitesse_Vent  Pluviometrie
3891              NaN       NaN           NaN           0.0
3894              NaN       NaN           NaN           0.0
3895              NaN       NaN           NaN           0.0
               Lieu  annee  mois  jour  Pression  Temperature_Max  \
2912  KANDI_AEROPORT   2022    12    22     980.1             30.0

      Temperature_Min  Humidite  Vitesse_Vent  Pluviometrie
2912             17.5      36.0           3.1           NaN
```

Certaines observations de Parakou présentent un taux très élevé de valeurs manquantes. Ces lignes ont été supprimées afin de garantir la qualité des données et de limiter l'introduction d'erreurs liées aux méthodes d'imputation.

Par ailleurs, une observation de Kandi contenant uniquement une valeur manquante pour la pluviométrie a également été supprimée compte tenu de son caractère isolé et de son impact négligeable sur l'ensemble des données.

```
Parakou=Parakou.dropna(thresh=6)
Kandi=Kandi.drop([Kandi[Kandi.Pluviometrie.isnull()].index[0]],axis=0)
```

4.6 Méthodes d'imputation

Plusieurs méthodes d'imputation ont été étudiées :

- moyenne historique ;
- médiane historique ;
- interpolation linéaire ;
- moyenne globale ;
- médiane globale.

4.6.1 Moyenne historique et médiane historique

Ces deux méthodes consistent à remplacer une valeur manquante respectivement par la moyenne et la médiane des valeurs observées des années précédentes, pour le même jour et le même mois.

4.6.2 Interpolation linéaire

Cette méthode consiste à remplacer une valeur manquante en utilisant la valeur précédente et la valeur suivante. Par exemple, en observant la pression du jour précédent et celle du jour suivant, il est possible d'estimer la pression du jour concerné.

4.6.3 Moyenne globale et médiane globale

Ces méthodes consistent à remplacer une valeur manquante respectivement par la moyenne et la médiane générales.

4.7 Protocole expérimental

Afin de comparer les différentes méthodes d'imputation, certaines valeurs connues ont été supprimées artificiellement dans les données.

Les valeurs supprimées ont ensuite été reconstruites par chaque méthode d'imputation.

Les performances ont été évaluées à l'aide des métriques :

- $MAE = \frac{1}{n}\sum_{i=1}^{n}|x_{i,pred} - x_{i,vrai}|$
- $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_{i,pred} - x_{i,vrai})^2}$.

4.8 Résultats de Kandi

```
print(df_resultat_Kandi)
```

	Variable	Méthode	MAE	RMSE
0	Pression	Moyenne historique	3.983393	6.825533
1	Pression	Médiane historique	2.236250	5.611945
2	Pression	Interpolation	0.707833	1.804480
3	Pression	Moyenne globale	2.813529	5.436928
4	Pression	Médiane globale	2.309000	5.378150
5	Humidite	Moyenne historique	7.518853	10.074710
6	Humidite	Médiane historique	7.490000	10.068515
7	Humidite	Interpolation	4.405000	6.464239
8	Humidite	Moyenne globale	18.836105	21.255002
9	Humidite	Médiane globale	18.820000	21.244764
10	Vitesse_Vent	Moyenne historique	0.616307	0.839723
11	Vitesse_Vent	Médiane historique	0.634500	0.866357
12	Vitesse_Vent	Interpolation	0.470000	0.618708
13	Vitesse_Vent	Moyenne globale	0.572254	0.780300
14	Vitesse_Vent	Médiane globale	0.573000	0.785557

4.9 Résultats de Parakou

```
print(df_resultat_Parakou)
```

	Variable	Méthode	MAE	RMSE
0	Pression	Moyenne historique	2.538769	6.741995
1	Pression	Médiane historique	1.785000	6.121136
2	Pression	Interpolation	0.396000	0.527926
3	Pression	Moyenne globale	2.075019	6.050059
4	Pression	Médiane globale	2.010000	6.086050
5	Temperature_Max	Moyenne historique	1.560396	1.984032
6	Temperature_Max	Médiane historique	1.463750	1.896808
7	Temperature_Max	Interpolation	1.106833	1.550974
8	Temperature_Max	Moyenne globale	2.844814	3.255498
9	Temperature_Max	Médiane globale	2.813000	3.241157
10	Temperature_Min	Moyenne historique	1.248569	1.610092
11	Temperature_Min	Médiane historique	1.282875	1.651182
12	Temperature_Min	Interpolation	0.855000	1.166598
13	Temperature_Min	Moyenne globale	1.317245	1.673103
14	Temperature_Min	Médiane globale	1.316000	1.672782
15	Humidite	Moyenne historique	9.173177	13.232393
16	Humidite	Médiane historique	9.355000	13.759270
17	Humidite	Interpolation	4.918333	6.776532
18	Humidite	Moyenne globale	17.474566	20.693217
19	Humidite	Médiane globale	17.090000	22.188961
20	Vitesse_Vent	Moyenne historique	0.622360	0.794235
21	Vitesse_Vent	Médiane historique	0.628250	0.802749
22	Vitesse_Vent	Interpolation	0.403000	0.541951
23	Vitesse_Vent	Moyenne globale	0.564437	0.707587
24	Vitesse_Vent	Médiane globale	0.552000	0.701142

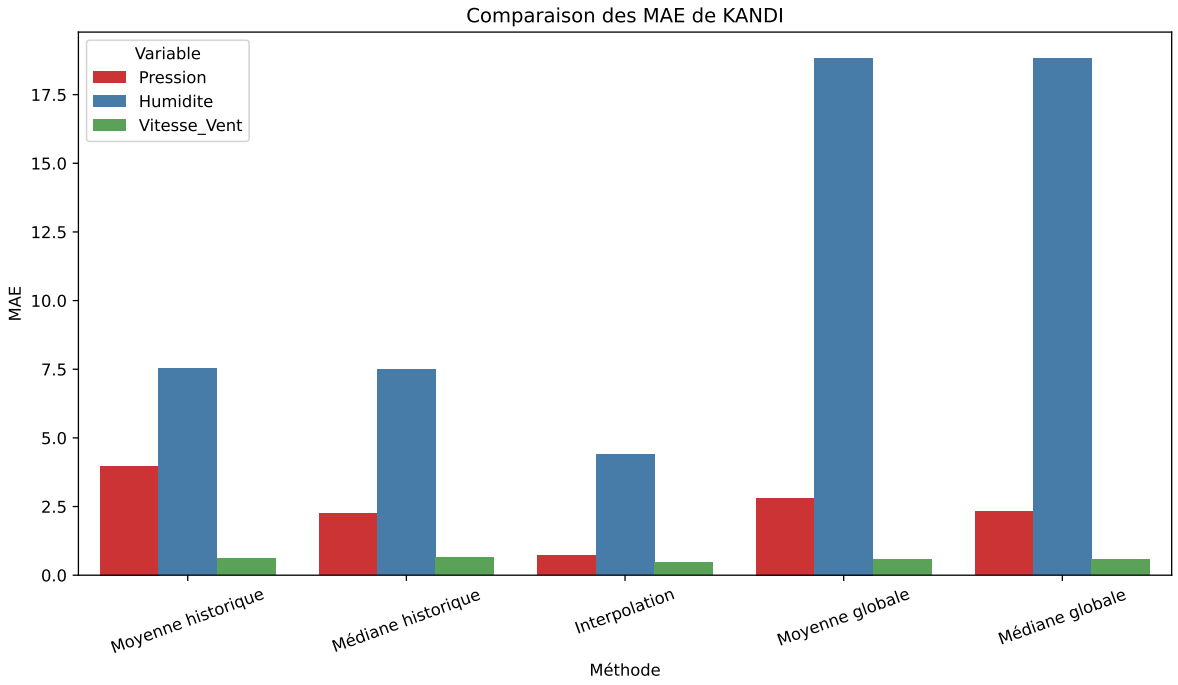
4.10 Représentation graphique des erreurs

4.10.1 Station de Kandi

```
plt.figure(figsize=(12,6))
sns.barplot(data=df_resultat_Kandi,
    x="Méthode",
    y="MAE",
    hue="Variable",
    palette="Set1"
)

plt.title("Comparaison des MAE de KANDI")
```

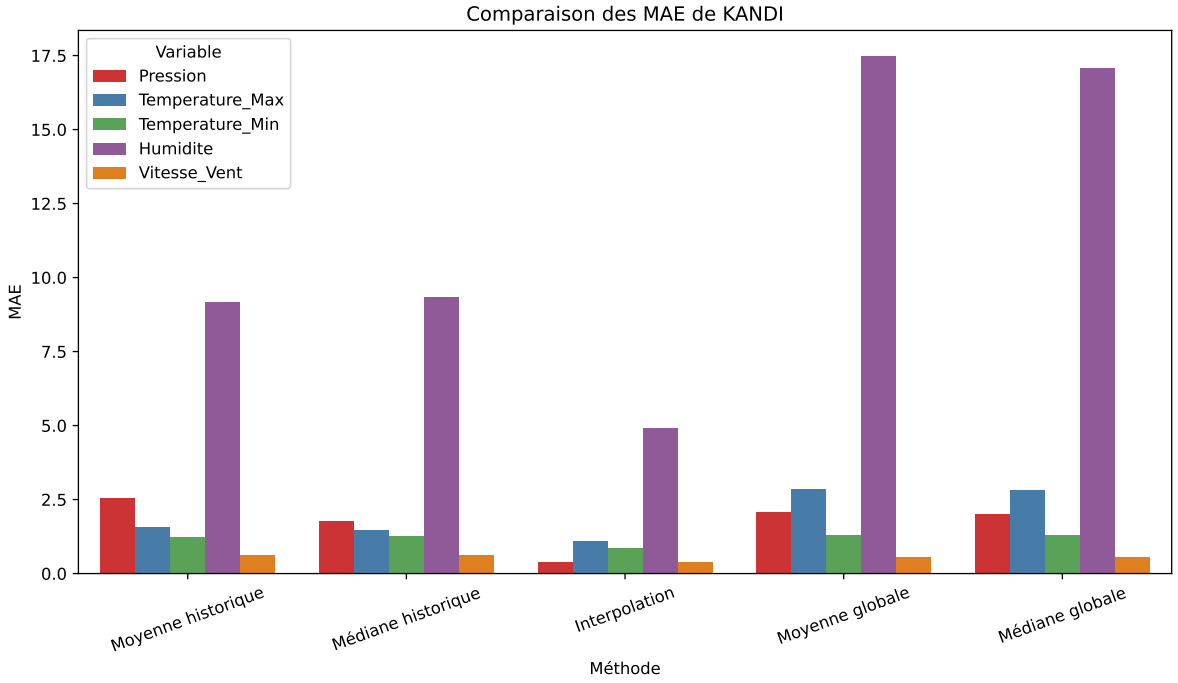
```
plt.xticks(rotation=20)
plt.show()
```



4.10.2 Station de Parakou

```
plt.figure(figsize=(12,6))
sns.barplot(data=df_resultat_Parakou,
    x="Méthode",
    y="MAE",
    hue="Variable",
    palette="Set1"
)

plt.title("Comparaison des MAE de KANDI")
plt.xticks(rotation=20)
plt.show()
```



5 Conclusion

Les résultats obtenus montrent que l'interpolation linéaire fournit les meilleures performances sur la majorité des variables météorologiques étudiées.

Cette méthode présente les plus faibles valeurs de MAE et de RMSE pour les stations de Kandi et de Parakou.

Les méthodes basées sur les moyennes et médianes globales présentent des performances moins satisfaisantes, ce qui montre qu'elles ne prennent pas suffisamment en compte la dynamique temporelle des données climatiques.

Cette étude a permis de structurer les données météorologiques, d'analyser les valeurs manquantes et de comparer plusieurs méthodes d'imputation.

l'interpolation linéaire sera utilisée dans la suite du projet pour le prétraitement des données avant l'analyse exploratoire et la modélisation prédictive.

6 Prétaitement final des données avec la méthode d'interpolation linéaire

```
Kandi_final = methode_interpo(Kandi, liste_variable_marquante_Kandi)
Parakou_final = methode_interpo(Parakou, liste_variable_marquante_Parakou)

Kandi_final.to_csv("data/Kandi_final.csv", index=False)
Parakou_final.to_csv("data/Parakou_final.csv", index=False)
print(Kandi_final.isnull().sum()[0<Kandi_final.isnull().sum()])
print(Parakou_final.isnull().sum()[0<Parakou_final.isnull().sum()])
```

```
Series([], dtype: int64)
Series([], dtype: int64)
```